# CHAPTER FOUR

## USE CASE MODELLING

### Common Modeling Techniques Modeling Different Views of a System

- When you model a system from different views, you are in effect constructing your system simultaneously from multiple dimensions.

**To model a system from different views**

- Decide which views you need to best express the architecture of your system and to expose the technical risks to your project
- For each of these views, decide which artifacts you need to create to capture the essential details of that view.
- As part of your process planning, decide which of these diagrams you'll want to put under some sort of formal or semi-formal control.
- For example, if you are modeling a simple monolithic application that runs on a single machine, you might need only the following handful of diagrams.

| | | |
|---|---|---|
| • Use case view | : | Use case diagrams |
| • Design view | : | Class diagrams (for structural modeling)<br>Interaction diagrams (for behavioral modeling) |
| • Process view | : | |
| • Implementation view | : | None required |
| • Deployment view | : | None required |

- If yours is a reactive system or if it focuses on process flow, you'll probably want to include state chart diagrams and activity diagrams, respectively, to model your system's behavior.
- Similarly, if yours is a client/server system, you'll probably want to include component diagrams and deployment diagrams to model the physical details of your system.
- Finally, if you are modeling a complex, distributed system, you'll need to employ the full range of the UML's diagrams in order to express the architecture of your system and the technical risks to your project, as in the following.

| | | |
|---|---|---|
| • Use case view | : | Use case diagrams Activity diagrams (for behavioral modeling) |
| • Design view | : | * Class diagrams (for structural modeling)<br>* Interaction diagrams (for behavioral modeling)<br>* Statechart diagrams (for behavioral modeling) |
| • Process view | : | * Class diagrams (for structural modeling)<br>* Interaction diagrams (for behavioral modeling) |
| • Implementation view | : | Component diagram |
| • Deployment view | : | Deployment diagrams |

**Modeling Different Levels of Abstraction**

- Not only do you need to view a system from several angles, you'll also find people involved in development who need the same view of the system but at different levels of abstraction
- Basically, there are two ways to model a system at different levels of abstraction:
    1. By presenting diagrams with different levels of detail against the same model
    2. By creating models at different levels of abstraction with diagrams that trace from one model to another.

**To model a system at different levels of abstraction by presenting diagrams with different levels of detail,**

- Consider the needs of your readers, and start with a given model
- If your reader is using the model to construct an implementation, she'll need diagrams that are at a lower level of abstraction which means that they'll need to reveal a lot of detail

- If she is using the model to present a conceptual model to an end user, she'll need diagrams that are at a higher level of abstraction which means that they'll hide a lot of detail
- Depending on where you land in this spectrum of low-to-high levels of abstraction, create a diagram at the right level of abstraction by hiding or revealing the following four categories of things from your model:

**Building blocks and relationships:**

- Hide those that are not relevant to the intent of your diagram or the needs of your reader.

**Adornments:**

- Reveal only the adornments of these building blocks and relationships that are essential to understanding your intent.

**Flow:**

- In the context of behavioral diagrams, expand only those messages or transitions that are essential to understanding your intent.

**Stereotypes:**

- In the context of stereotypes used to classify lists of things, such as attributes and operations, reveal only those stereotyped items that are essential to understanding your intent.

- The **main advantage** of this approach is that you are always modeling from a common semantic repository.

- The **main disadvantage** of this approach is that changes from diagrams at one level of abstraction may make obsolete diagrams at a different level of abstraction.

**To model a system at different levels of abstraction by creating models at different levels of abstraction,**

- Consider the needs of your readers and decide on the level of abstraction that each should view, forming a separate model for each level.
- In general, populate your models that are at a high level of abstraction with simple abstractions and your models that are at a low level of abstraction with detailed abstractions. Establish trace dependencies among the related elements of different models.
- In practice, if you follow the five views of an architecture, there are four common situations you'll encounter when modeling a system at different levels of abstraction:

**Use cases and their realization:**

- Use cases in a use case model will trace to collaborations in a design model.

**Collaborations and their realization:**

- Collaborations will trace to a society of classes that work together to carry out the collaboration.

**Components and their design:**

- Components in an implementation model will trace to the elements in a design model.

**Nodes and their components:**

- Nodes in a deployment model will trace to components in an implementation model.

The **main advantage** of the approach is that diagrams at different levels of abstraction remain more loosely coupled. This means that changes in one model will have less direct effect on other models.

The **main disadvantage** of this approach is that you must spend resources to keep these models and their diagrams synchronized

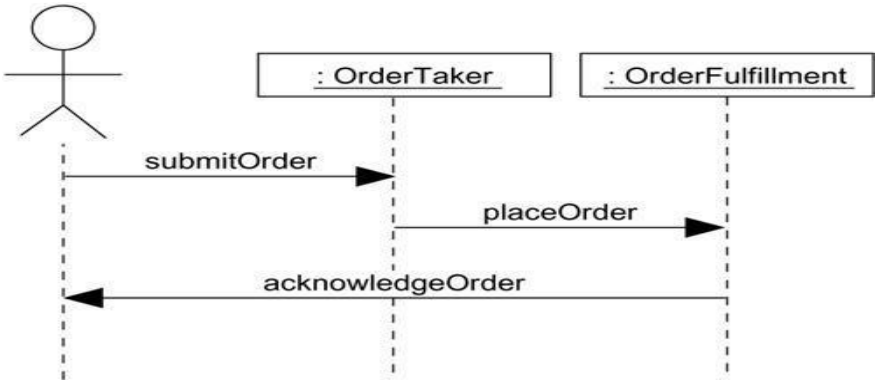



**Fig: Interaction Diagram at a High Level of Abstraction**

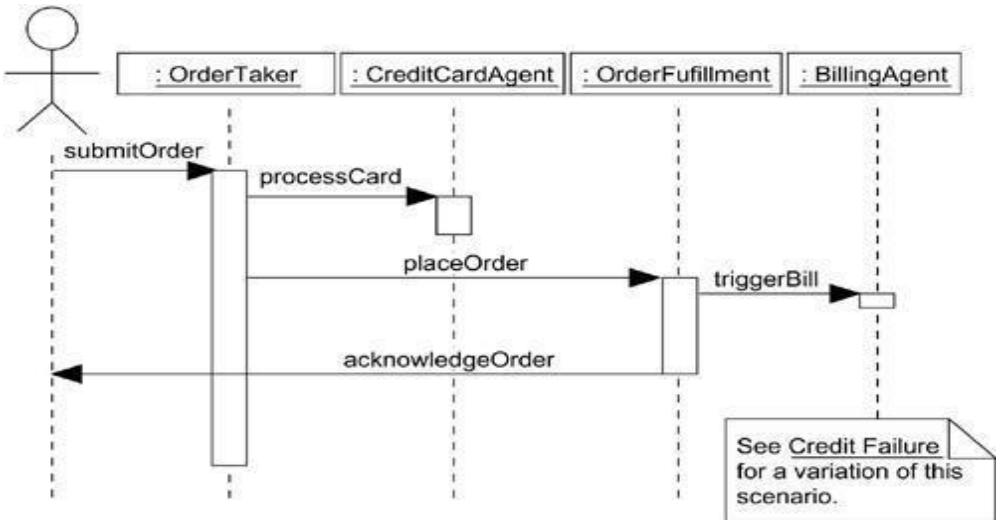



**Fig: Interaction at a Low Level of Abstraction**

* Both of these diagrams work against the same model, but at different levels of detail.

**Modeling Complex Views To model complex views**

- First, convince yourself there's no meaningful way to present this information at a higher level of abstraction, perhaps eliding some parts of the diagram and retaining the detail in other parts.
- If you've hidden as much detail as you can and your diagram is still complex, consider grouping some of the elements in packages or in higher level collaborations, then render only those packages or collaborations in your diagram.
- If your diagram is still complex, use notes and color as visual cues to draw the reader's attention to the points you want to make.

- If your diagram is still complex, print it in its entirety and hang it on a convenient large wall. You lose the interactivity an online version of the diagram brings, but you can step back from the diagram and study it for common patterns.

**Advanced Structural Modeling Advanced Classes**

- Classes are indeed the most important building block of any object-oriented system. However, classes are just one kind of an even more general building block in the UML• classifiers.
- A classifier is a mechanism that describes structural and behavioral features. Classifiers include classes, interfaces, datatypes, signals, components, nodes, use cases, and subsystems.
- Classifiers (and especially classes) have a number of advanced features beyond the simpler properties of attributes and operations described in the previous section: You can model multiplicity, visibility, signatures, polymorphism, and other characteristics.
- The UML provides a representation for a number of advanced properties, as Figure shows. This notation permits you to visualize, specify, construct, and document a class to any level of detail you wish, even sufficient to support forward and reverse engineering of models and code.

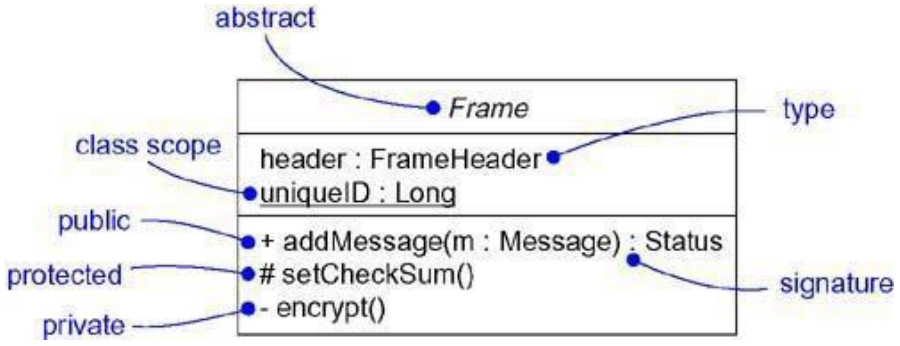



**Fig: Advanced Classes**

**Terms and Concepts**

- A classifier is a mechanism that describes structural and behavioral features. Classifiers include classes, interfaces, datatypes, signals, components, nodes, use cases, and subsystems.
- The most important kind of classifier in the UML is the class. A class is a description of a set of objects that share the same attributes, operations,

relationships, and semantics. Classes are not the only kind of classifier, however.

- The UML provides a number of other kinds of classifiers to help you model.

| | |
|---|---|
| • Interface | A collection of operations that are used to specify a service of a class or a component |
| • Datatype | A type whose values have no identity, including primitive built-in types (such as numbers and strings), as well as enumeration types (such as Boolean) |
| • Signal | The specification of an asynchronous stimulus communicated between instances |
| • Component | A physical and replaceable part of a system that conforms to and provides the realization of a set of interfaces |
| • Node | A physical element that exists at run time and that represents a computational resource, generally having at least some memory and often processing capability |
| • Use case | A description of a set of a sequence of actions, including variants, that a system performs that yields an observable result of value to a particular actor |
| • Subsystem | A grouping of elements of which some constitute a specification of the behavior offered by the other contained elements |

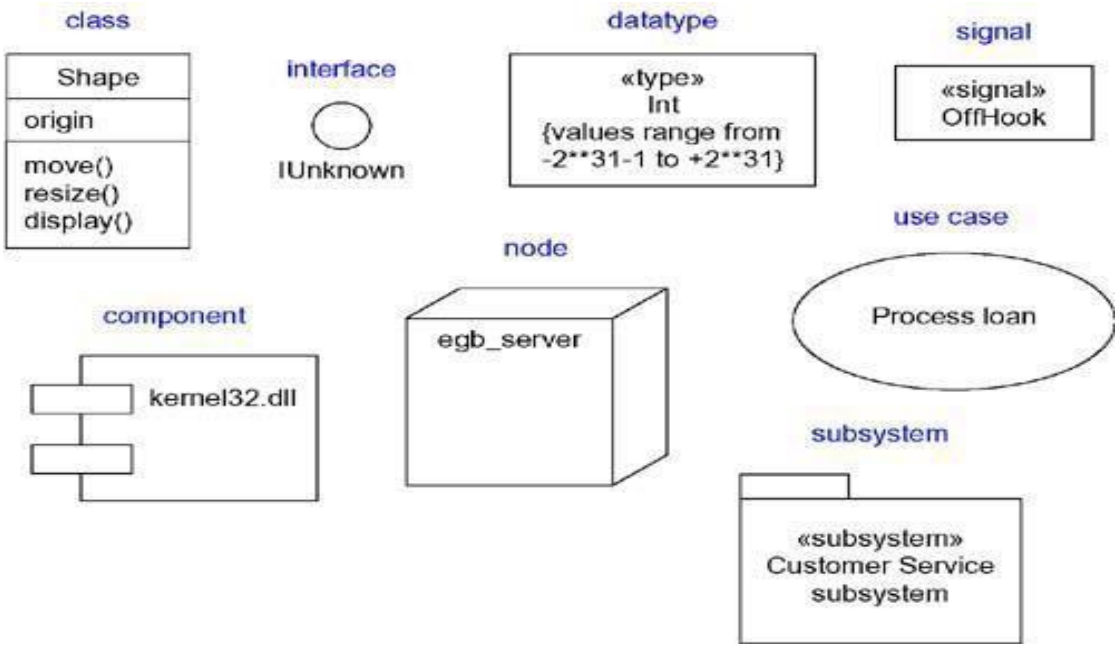



**Fig: Classifiers**

**Visibility**

- One of the most important details you can specify for a classifier's attributes and operations is its visibility.
- The visibility of a feature specifies whether it can be used by other classifiers. In the
- UML, you can specify any of three levels of visibility.

| | |
|---|---|
| 1. public | Any outside classifier with visibility to the given classifier can use the feature; specified by prepending the symbol + |
| 2. protected | Any descendant of the classifier can use the feature; specified by prepending the symbol # |
| 3. private | Only the classifier itself can use the feature; specified by prepending the symbol |

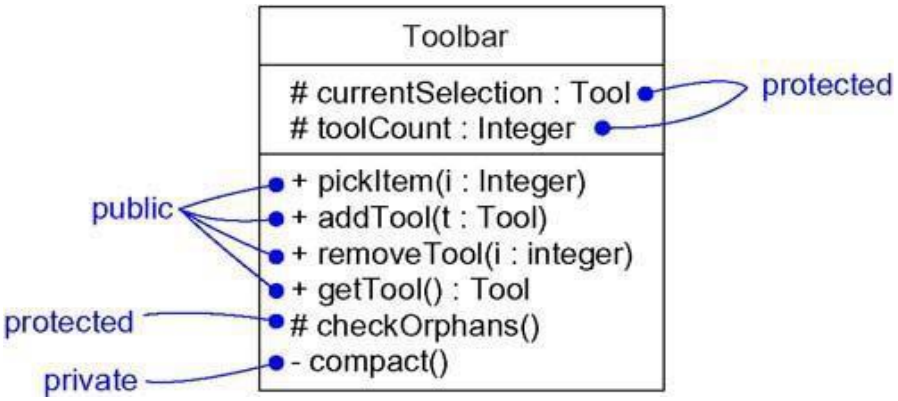



**Fig: Visibility**

The above Figure shows a mix of public, protected, and private figures for the class Toolbar.

**Scope**

- Another important detail you can specify for a classifier's attributes and operations is its owner scope.
- The owner scope of a feature specifies whether the feature appears in each instance of the classifier or whether there is just a single instance of the feature for all instances of the classifier.
- In the UML, you can specify two kinds of owner scope.

| | |
|---|---|
| 1. instance | Each instance of the classifier holds its own value for the feature. |
| 2. classifier | There is just one value of the feature for all instances of the classifier. |

As Figure (a simplification of the first figure) shows, a feature that is classifier scoped is rendered by underlining the feature's name. No adornment means that the feature is instance scoped.

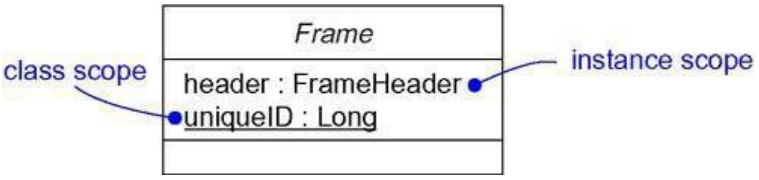
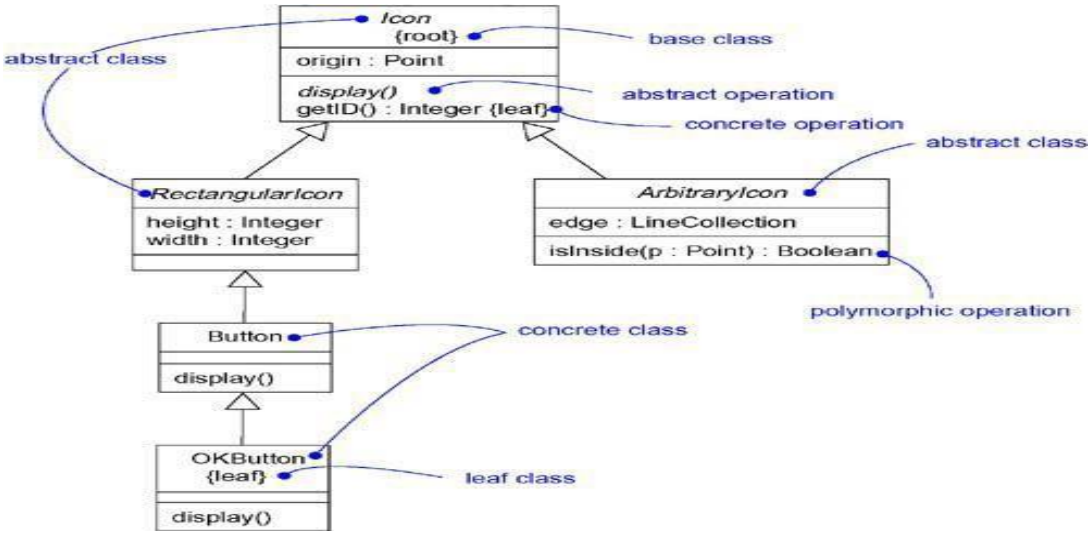



**Fig: Owner Scope Abstract, Root, Leaf, and Polymorphic Elements**

- You use generalization relationships to model a lattice of classes, with more-generalized
- Abstractions at the top of the hierarchy and more-specific ones at the bottom.
- Within these hierarchies, it's common to specify that certain classes are abstract• meaning that they may not have any direct instances.
- In the UML, you specify that a class is abstract by writing its name in italics. For example, as figure below shows, Icon, RectangularIcon, and




ArbitraryIcon are all abstract classes. By contrast, a concrete class (such as Button and OKButton) is one that may have direct instances.

**Fig: Abstract and Concrete Classes and Operations**

**Multiplicity**

- The number of instances a class may have is called its multiplicity. Multiplicity is a specification of the range of allowable cardinalities an entity may assume.
- In the UML, you can specify the multiplicity of a class by writing a multiplicity expression in the upper-right corner of the class icon.
- For example, in Figure NetworkController is a singleton class. Similarly, there are exactly three instances of the class ControlRod in the system.

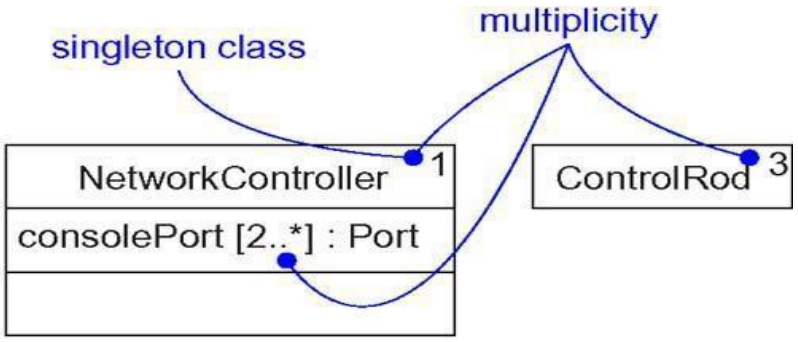



**Fig: Multiplicity**

**Attributes**

- At the most abstract level, when you model a class's structural features (that is, its attributes), you simply write each attribute's name.
- That's usually enough information for the average reader to understand the intent of your model.
- You can also specify the visibility, scope, and multiplicity of each attribute. There's still more. You can also specify the type, initial value, and changeability of each attribute.

In its full form, the syntax of an attribute in the UML is

[Visibility] name [multiplicity] [: type] [= initial-value] [{property-string}] For example, the following are all legal attribute declarations:

| | |
|---|---|
| • origin | Name only |
| • + origin | Visibility and name |
| • origin : Point | Name and type |
| • head : *Item | Name and complex type |
| • name [0..1] : String | Name, multiplicity, and type |
| • origin : Point = (0,0) | Name, type, and initial value |
| • id : Integer {frozen} | Name and property |

There are three defined properties that you can use with attributes.

| | |
|---|---|
| 1. changeable | There are no restrictions on modifying the attribute's value. |
| 2. addOnly | For attributes with a multiplicity greater than one, additional values may be added, but once created, a value may not be removed or altered. |
| 3. frozen | The attribute's value may not be changed after the object is initialized. |

**Operations**

- At the most abstract level, when you model a class's behavioral features (that is, its operations and its signals), you will simply write each operation's name
- You can also specify the visibility and scope of each operation.
- You can also specify the parameters, return type, concurrency semantics, and other properties of each operation. Collectively, the name of an operation plus its parameters (including its return type, if any) is called the operation's signature.
- The UML distinguishes between operation and method.
- An operation specifies a Service that can be requested from any object of the class to affect behavior;
- a method is an implementation of an operation. Every non abstract operation of a class must have a method, which supplies an executable algorithm as a body.

In its full form, the syntax of an operation in the UML is

[Visibility] name [(parameter-list)] [: return-type] [{property-string}] For example, the following are all legal operation declarations:

| | |
|---|---|
| • `display` | Name only |
| • `+ display` | Visibility and name |
| • `set(n : Name, s : String)` | Name and parameters |
| • `getID() : Integer` | Name and return type |
| • `restart() {guarded}` | Name and property |

In an operation's signature, you may provide zero or more parameters, each of which follows the

Syntax [direction] name : type [= default-value]

Direction may be any of the following values:

| | |
|---|---|
| • `in` | An input parameter; may not be modified |
| • `out` | An output parameter; may be modified to communicate information to the caller |
| • `inout` | An input parameter; may be modified |

In addition to the leaf property described earlier, there are four defined properties that you can use with operations

| | |
|---|---|
| 1. `isQuery` | Execution of the operation leaves the state of the system unchanged. In other words, the operation is a pure function that has no side effects. |
| 2. `sequential` | Callers must coordinate outside the object so that only one flow is in the object at a time. In the presence of multiple flows of control, the semantics and integrity of the object cannot be guaranteed. |
| 3. `guarded` | The semantics and integrity of the object is guaranteed in the presence of multiple flows of control by sequentializing all calls to all of the object's guarded operations. In effect, exactly one operation at a time can be invoked on the object, reducing this to sequential semantics. |
| 4. `concurrent` | The semantics and integrity of the object is guaranteed in the presence of multiple flows of control by treating the operation as atomic. Multiple calls from concurrent flows of control may occur simultaneously to one object on any concurrent operation, and all may proceed concurrently with correct semantics; |

**Template Classes**

- A template is a parameterized element. In such languages as C++ and Ada, you can write template classes, each of which defines a family of classes (you can also write template functions, each of which defines a family of functions).
- A template includes slots for classes, objects, and values, and these slots serve as the template's parameters.
- You can't use a template directly; you have to instantiate it first. Instantiation involves binding these formal template parameters to actual ones.
- For a template class, the result is a concrete class that can be used just like any ordinary class.
- The most common use of template classes is to specify containers that can be instantiated for specific elements, making them type-safe.

For example, the following C++ code fragment declares a parameterized Map class. template<class Item, class Value, int Buckets>

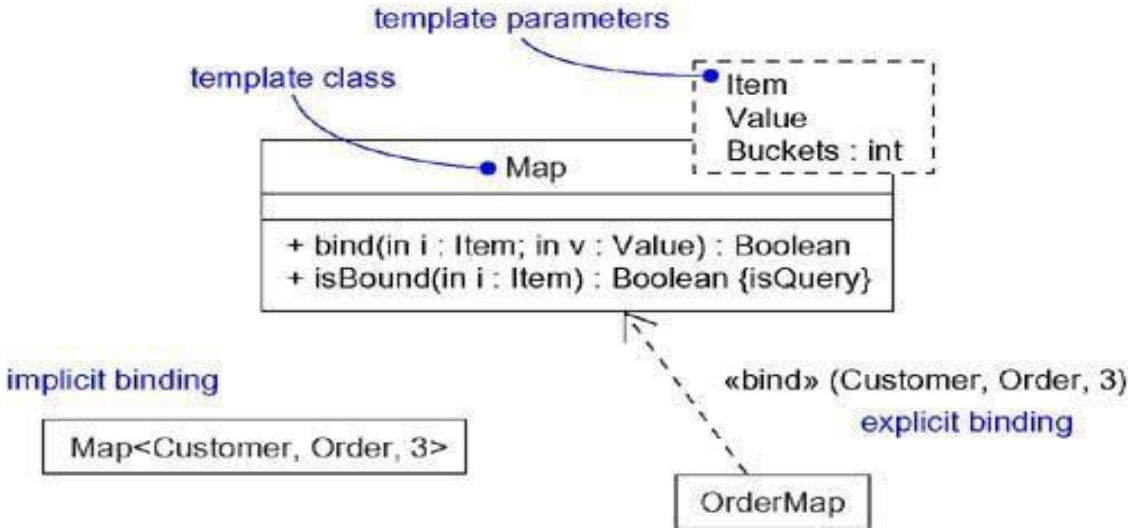
class Map { public: virtual Boolean bind(const Item&, const Value&); virtual Boolean isBound(const Item&) const;

...

};

You might then instantiate this template to map Customer objects to Order objects. m: Map<Customer, Order, 3>;

You can model template classes in the UML as well. As Figure shows, you render a Template class just as you do an ordinary class, but with an additional dashed box in




the upper right corner of the class icon, which lists the template parameters.

**Fig: Template Classes**

**Standard Elements**

- All of the UML's extensibility mechanisms apply to classes. Most often, you'll use tagged values to extend class properties (such as specifying the version of a class) and stereotypes to specify new kinds of components (such as model- specific components).
- The UML defines four standard stereotypes that apply to classes.

| | |
|---|---|
| 1. metaclass | Specifies a classifier whose objects are all classes |
| 2. powertype | Specifies a classifier whose objects are the children of a given parent |
| 3. stereotype | Specifies that the classifier is a stereotype that may be applied to other elements |
| 4. utility | Specifies a class whose attributes and operations are all class scoped |

**Common Modeling Techniques Modeling the Semantics of a Class**

To model the semantics of a class, choose among the following possibilities, arranged from informal to formal.

- Specify the responsibilities of the class. A responsibility is a contract or obligation of a type or class and is rendered in a note (stereotyped as

responsibility) attached to the class, or in an extra compartment in the class icon.

- Specify the semantics of the class as a whole using structured text, rendered in a note (stereotyped as semantics) attached to the class.
- Specify the body of each method using structured text or a programming language, rendered in a note attached to the operation by a dependency relationship.
- Specify the pre- and post-conditions of each operation, plus the invariants of the class as a whole, using structured text. These elements are rendered in notes (stereotyped as precondition, postcondition, and invariant) attached to the operation or class by a dependency relationship.
- Specify a state machine for the class. A state machine is a behavior that specifies the sequences of states an object goes through during its lifetime in response to events, together with its responses to those events.
- Specify a collaboration that represents the class. A collaboration is a society of roles and other elements that work together to provide some cooperative behavior that's bigger than the sum of all the elements. A collaboration has a structural part, as well as a dynamic part, so you can use collaborations to specify all dimensions of a class's semantics.
- Specify the pre- and post-conditions of each operation, plus the invariants of the class as a whole, using a formal language such as OCL.

**Advanced Relationships**

- A relationship is a connection among things. In object-oriented modeling, the four most important relationships are dependencies, generalizations, associations, and realizations.
- Graphically, a relationship is rendered as a path, with different kinds of lines used to distinguish the different relationships.

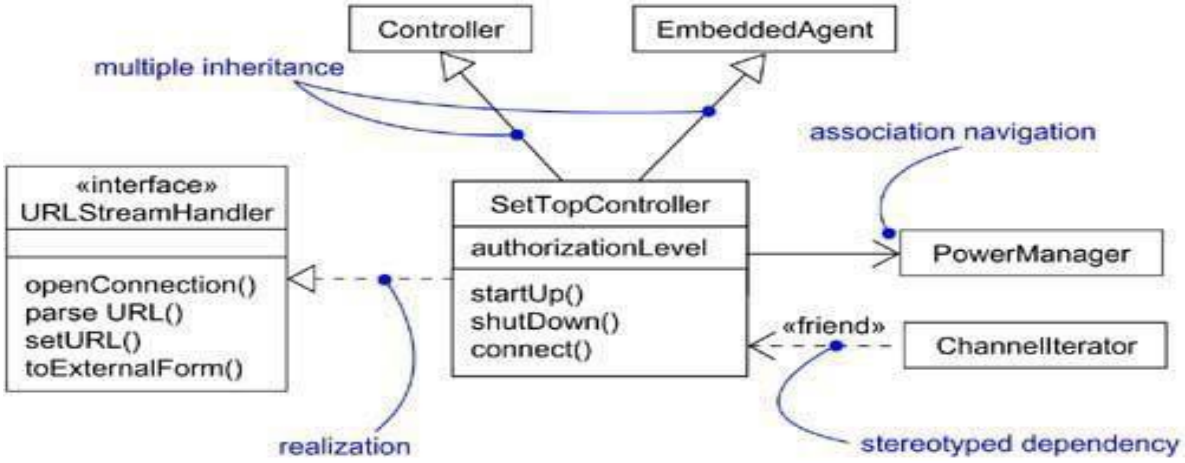



**Fig: Advanced Relationships**

**Terms and Concepts**

- A relationship is a connection among things. In object-oriented modeling, the four most important relationships are dependencies, generalizations, associations, and realizations.
- Graphically, a relationship is rendered as a path, with different kinds of lines used to distinguish the different relationships.

**Dependency**

- A dependency is a using relationship, specifying that a change in the specification of one thing may affect another thing that uses it, but not necessarily the reverse. Graphically, a dependency is rendered as a dashed line
- A plain, unadorned dependency relationship is sufficient for most of the using relationships you'll encounter. However, if you want to specify a shade of meaning, the UML defines a number of stereotypes that may be applied to dependency relationships.
- There are 17 such stereotypes, all of which can be organized into six groups.
- First, there are eight stereotypes that apply to dependency relationships among classes and objects in class diagrams.

| | | |
|---|---|---|
| **1** | **bind** | Specifies that the source instantiates the target template using the given<br>actual parameters |
| **2** | **derive** | Specifies that the source may be computed from the target |
| **3** | **friend** | Specifies that the source is given special visibility into the target |
| **4** | **instanceOf** | Specifies that the source object is an instance of the target classifier |
| **5** | **instantiate** | Specifies that the source creates instances of the target |
| **6** | **powertype** | Specifies that the target is a powertype of the source; a powertype is a<br>classifier whose objects are all the children of a given parent |
| **7** | **refine** | Specifies that the source is at a finer degree of abstraction than the target |
| **8** | **use** | Specifies that the semantics of the source element depends on the<br>semantics of the public part of the target |

**bind:-**bind includes a list of actual arguments that map to the formal arguments of the template.

**derive-**When you want to model the relationship between two attributes or two associations, one of which is concrete and the other is conceptual.

**friend-**When you want to model relationships such as found with C++ friend classes.

**instanceof-**When you want to model the relationship between a class and an object in the same diagram, or between a class and its metaclass.

**instantiate-**when you want to specify which element creates objects of another.

**powertype-**when you want to model classes that cover other classes, such as you'll find when modeling databases

**refine-**when you want to model classes that are essentially the same but at different levels of abstraction.

**use-**when you want to explicitly mark a dependency as a using relationship

* There are **two** stereotypes that apply to dependency relationships **among packages.**

| | | |
|---|---|---|
| **9** | **access** | Specifies that the source package is granted the right to reference the elements of the target package |
| **10** | **import** | A kind of access that specifies that the public contents of the target package enter the flat namespace of the source, as if they had been declared in the source |

* **Two** stereotypes apply to dependency relationships among **use cases**:

| | | |
|---|---|---|
| **11** | **extend** | Specifies that the target use case extends the behavior of the source |
| **12** | **include** | Specifies that the source use case explicitly incorporates the behavior of another use case at a location specified by the source |

* There are **three** stereotypes when modeling interactions among **objects.**

| | | |
|---|---|---|
| **13** | **become** | Specifies that the target is the same object as the source but at a later point in time and with possibly different values, state, or roles |
| **14** | **call** | Specifies that the source operation invokes the target operation |
| **15** | **copy** | Specifies that the target object is an exact, but independent, copy of the source |

* We'll use become and copy when you want to show the role, state, or attribute value of one object at different points in time or space
* You'll use call when you want to model the calling dependencies among operations.

* **One** stereotype you'll encounter in the context of **state machines** is

| | | |
|---|---|---|
| **16** | **send** | Specifies that the source operation sends the target event |

* We'll use send when you want to model an operation dispatching a given event to a target object.

* The send dependency in effect lets you tie independent state machines together. Finally**, one** stereotype that you'll encounter in the context of organizing the elements of your system into subsystems and models is

| | | |
|---|---|---|
| **17** | **trace** | Specifies that the target is an historical ancestor of the source |

* We'll use trace when you want to model the relationships among elements in different models

**Generalization**

- A generalization is a relationship between a general thing (called the superclass or parent) and a more specific kind of that thing (called the subclass or child).
- In a generalization relationship, instances of the child may be used anywhere instances of the parent apply—meaning that the child is substitutable for the parent.

- A plain, unadorned generalization relationship is sufficient for most of the inheritance relationships you'll encounter. However, if you want to specify a shade of meaning.

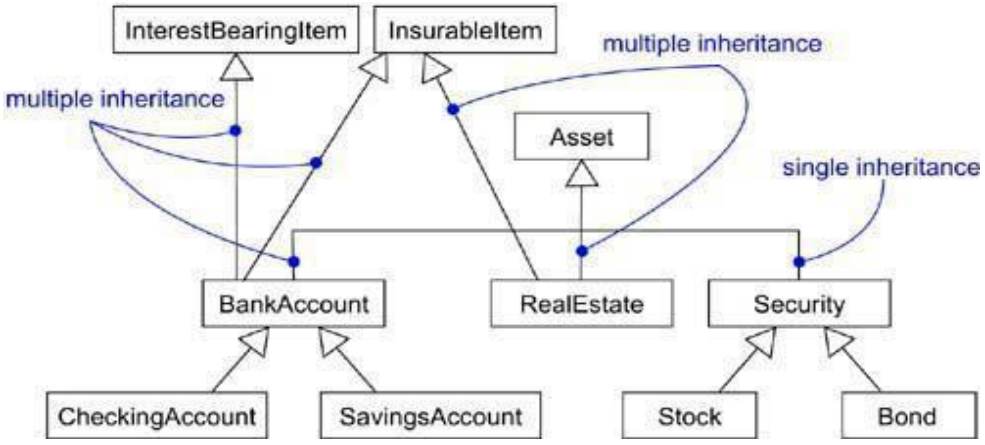



**Fig: Multiple Inheritance**

- For example, the above figure shows a set of classes drawn from a financial services application. You see the class Asset with three children: BankAccount, RealEstate, and Security. Two of these children (BankAccount and Security) have their own children.
- For example, Stock and Bond are both children of Security. Two of these children (BankAccount and RealEstate) inherit from multiple parents. RealEstate, for example, is a kind of Asset, as well as a kind of InsurableItem, and BankAccount is a kind of Asset, as well as a kind of InterestBearingItem and an InsurableItem.
- The UML defines one stereotype and four constraints that may be applied to generalization relationships.

| | | |
|---|---|---|
| **1** | **implementation** | Specifies that the child inherits the implementation of the parent but does not make public nor support its interfaces, thereby violating substitutability |

**implementation**

- We'll use implementation when you want to model private inheritance, such as found in C++.
- Next, there are four standard constraints that apply to generalization relationships

| | | |
|---|---|---|
| **1** | **complete** | Specifies that all children in the generalization have been specified in the model and that no additional children are permitted |
| **2** | **incomplete** | Specifies that not all children in the generalization have been specified (even if some are elided) and that additional children are permitted |

| | | |
|---|---|---|
| **3** | **disjoint** | Specifies that objects of the parent may have no more than one of the<br>children as a type |
| **4** | **overlapping** | Specifies that objects of the parent may have more than one of the<br>children as a type |

**Complete**

- We'll use the complete constraint when you want to show explicitly that you've fully specified a hierarchy in the model (although no one diagram may show that hierarchy);

**Incomplete**

- We'll use incomplete to show explicitly that you have not stated the full specification of the hierarchy in the model (although one diagram may show everything in the model).

**Disjoint & overlapping**

- These two constraints apply only in the context of multiple inheritance.
- We'll use disjoint and overlapping when you want to distinguish between static classification (disjoint) and dynamic classification (overlapping).

**Association**

- An association is a structural relationship, specifying that objects of one thing are connected to objects of another.
- We use associations when you want to show structural relationships.
- There are four basic adornments that apply to an association: a name, the role at each end of the association, the multiplicity at each end of the association, and aggregation.
- For advanced uses, there are a number of other properties you can use to model subtle details, such as

Navigation Vision Qualification

Various flavors of aggregation.

**Navigation**

- Unadorned association between two classes, such as Book and Library, it's possible to navigate from objects of one kind to objects of the other kind. Unless otherwise specified, navigation across an association is bidirectional.
- However, there are some circumstances in which you'll want to limit navigation to just one direction.

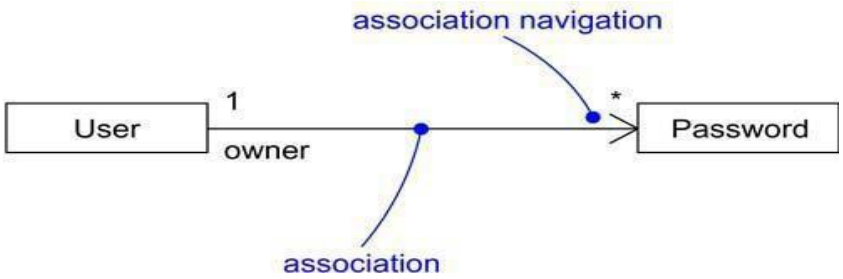



**Fig: Navigation**

- The above figure shows, when modeling the services of an operating system, you'll find an association between User and Password objects.
- Given a User, you'll want to be able to find the corresponding Password objects; but given a Password, you don't want to be able to identify the corresponding User.
- You can explicitly represent the direction of navigation by adorning an association with an arrowhead pointing to the direction of traversal.

**Visibility**

- Given an association between two classes, objects of one class can see and navigate to objects of the other, unless otherwise restricted by an explicit statement of navigation.
- However, there are circumstances in which you'll want to limit the visibility across that association relative to objects outside the association.
- In the UML, you can specify three levels of visibility for an association end, just as you can for a class's features by appending a visibility symbol to a role name the visibility of a role is public.
- Private visibility indicates that objects at that end are not accessible to any objects outside the association.

- Protected visibility indicates that objects at that end are not accessible to any objects outside the association, except for children of the other end.

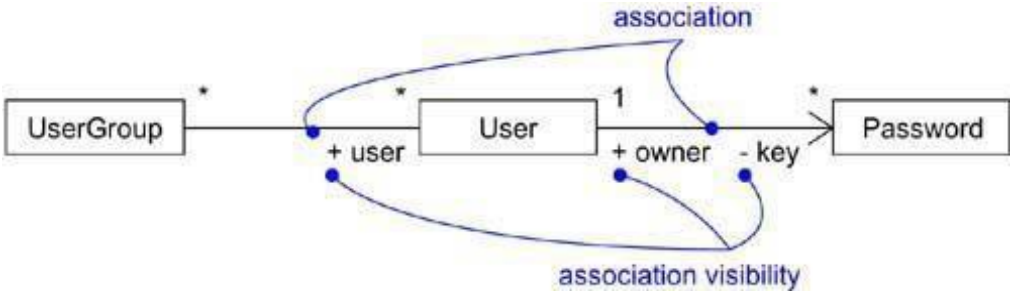



**Fig: Visibility**

**Qualification**

- In the context of an association, one of the most common modeling idioms you'll encounter is the problem of lookup. Given an object at one end of an association, how do you identify an object or set of objects at the other end.
- In the UML, you'd model this idiom using a qualifier, which is an association attribute whose values partition the set of objects related to an object across an association.
- You render a qualifier as a small rectangle attached to the end of an association, placing the attributes in the rectangle.

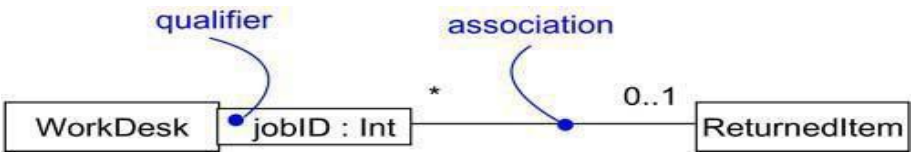



**Fig: Qualification**

- The above figure shows, you'd model an association between two classes, WorkDesk and ReturnedItem.
- In the context of the WorkDesk, you'd have a jobId that would identify a particular ReturnedItem. In that sense, jobId is an attribute of the association.
- It's not a feature of ReturnedItem because items really have no knowledge of things like repairs or jobs.
- An object of WorkDesk and given a particular value for jobId, you can navigate to zero or one objects of ReturnedItem

**Interface Specifier**

- An interface is a collection of operations that are used to specify a service of a class or a component
- Collectively, the interfaces realized by a class represent a complete specification of the behavior of that class.
- However, in the context of an association with another target class, a source class may choose to present only part of its face to the world.

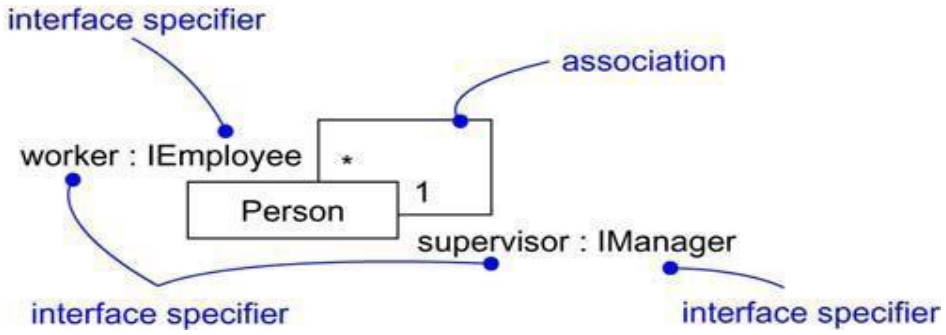



**Fig: Interface Specifiers**

- A Person class may realize many interfaces: IManager, IEmployee, IOfficer, and so on you can model the relationship between a supervisor and her workers with a one-to-many association, explicitly labeling the roles of this association as supervisor and worker.
- In the context of this association, a Person in the role of supervisor presents only the IManager face to the worker; a Person in the role of worker presents only the IEmployee face to the supervisor. As the figure shows, you can explicitly show the type of role using the syntax rolename : iname, where iname is some interface of the other classifier.

**Composition**

- Simple aggregation is entirely conceptual and does nothing more than distinguish a "whole" from a "part."
- Composition is a form of aggregation, with strong ownership and coincident lifetime as part of the whole.
- Parts with non-fixed multiplicity may be created after the composite itself, but once created they live and die with it. Such parts can also be explicitly removed before the death of the composite.

- This means that, in a composite aggregation, an object may be a part of only one composite at a time
- In addition, in a composite aggregation, the whole is responsible for the disposition of its parts, which means that the composite must manage the creation and destruction of its parts

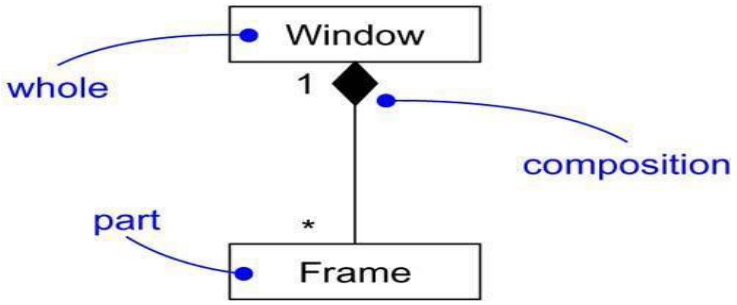



**Fig: Composition**

- The above figure shows, composition is really just a special kind of association and is specified by adorning a plain association with a filled diamond at the whole end.

**Association Classes**

- In an association between two classes, the association itself might have properties.
- An association class can be seen as an association that also has class properties, or as a class that also has association properties.
- We render an association class as a class symbol attached by a dashed line to an association.

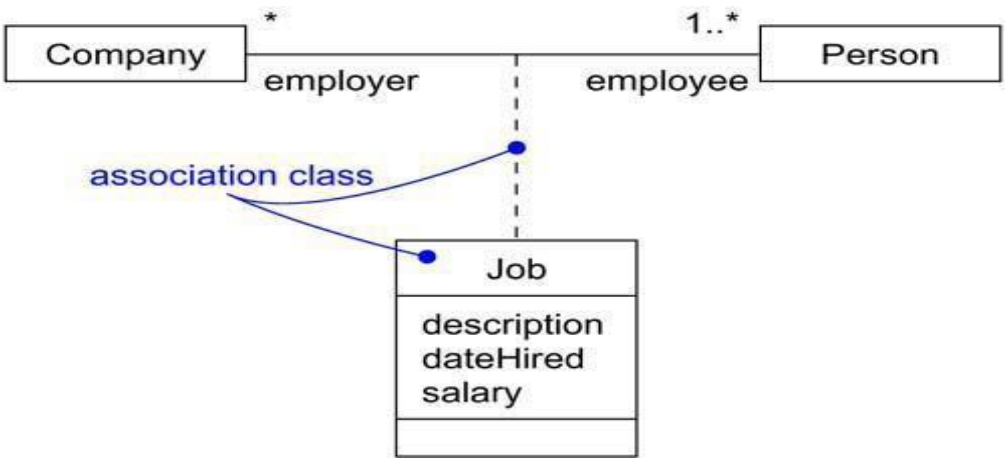


**Fig: Association Classes Constraints**

* UML defines **five** constraints that may be applied to association relationships.

| | | |
|---|---|---|
| **1** | **Implicit** | Specifies that the relationship is not manifest but, rather, is only conceptual |
| **2** | **Ordered** | Specifies that the set of objects at one end of an association are in an explicit order |
| **3** | **changeable** | Links between objects may be added, removed, and changed freely |
| **4** | **addOnly** | New links may be added from an object on the opposite end of the |
| | | association |
| **5** | **Frozen** | A link, once added from an object on the opposite end of the association, may not be modified or deleted |

**Implicit**

- If you have an association between two base classes, you can specify that same association between two children of those base classes
- You can specify that the objects at one end of an association (with a multiplicity greater than one) are ordered or unordered.

**Ordered**

- For example, in a User/Password association, the Passwords associated with the User might be kept in a least-recently used order, and would be marked as ordered.

Finally, there is **one** constraint for managing related sets of associations:

| | | |
|---|---|---|
| **1** | **xor** | Specifies that, over a set of associations, exactly one is manfest for each associated object |

**Realization**

- Realization is sufficiently different from dependency, generalization, and association relationships that it is treated as a separate kind of relationship.

- A realizations a semantic relationship between classifiers in which one classifier specifies a contract that another classifier guarantees to carry out.
- Graphically, a realization is rendered as a dashed directed line with a large open arrowhead pointing to the classifier that specifies the contract.
- You'll use realization in two circumstances: in the context of interfaces and in the context of collaborations
- Most of the time, you'll use realization to specify the relationship between an interface and the class or component that provides an operation or service for it
- You'll also use realization to specify the relationship between a use case and the collaboration that realizes that use case.

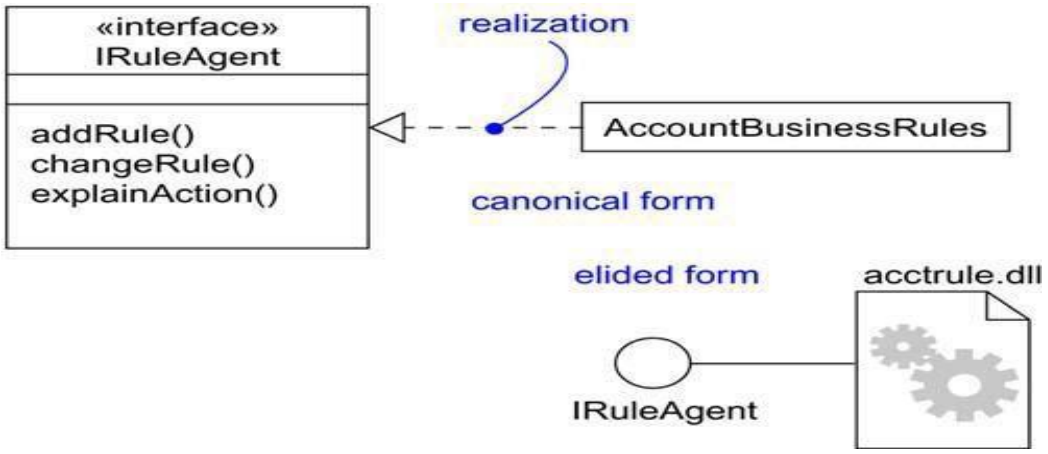



**Fig: Realization of an Interface**

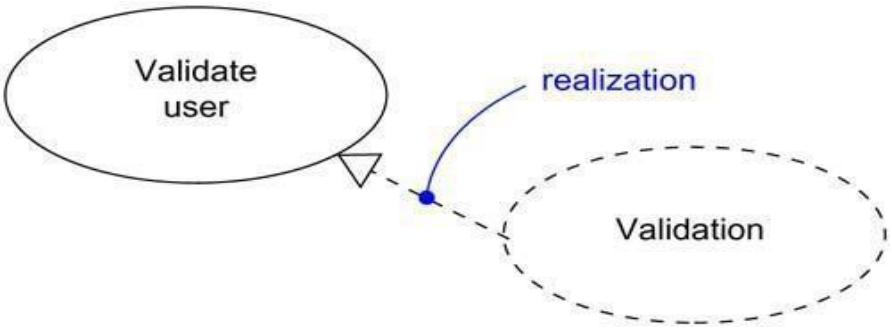



**Fig: Realization of a Use Case Common Modeling Techniques**

**Modeling Webs of Relationships**

- When you model the vocabulary of a complex system, you may encounter dozens, if not hundreds or thousands, of classes, interfaces, components, nodes, and use cases.
- Establishing a crisp boundary around each of these abstractions is hard
- This requires you to form a balanced distribution of responsibilities in the system as a whole, with individual abstractions that are tightly cohesive and with relationships that are expressive, yet loosely coupled

**To model webs of relationships**

- Don't begin in isolation. Apply use cases and scenarios to drive your discovery of the relationships among a set of abstractions.
- In general, start by modeling the structural relationships that are present. These reflect the static view of the system and are therefore fairly tangible.
- Next, identify opportunities for generalization/specialization relationships; use multiple inheritance sparingly.
- Only after completing the preceding steps should you look for dependencies; they generally represent more-subtle forms of semantic connection.
- For each kind of relationship, start with its basic form and apply advanced features only as absolutely necessary to express your intent.
- Remember that it is both undesirable and unnecessary to model all relationships among a set of abstractions in a single diagram or view. Rather, build up your system's relationships by considering different views on the system. Highlight interesting sets of relationships in individual diagrams.

**Interfaces, type and roles**

- In the UML, you use interfaces to model the seams in a system.
- An interface is a collection of operations used to specify a service of a class or a component.
- By declaring an interface, you can state the desired behavior of an abstraction independent of an implementation of that abstraction.

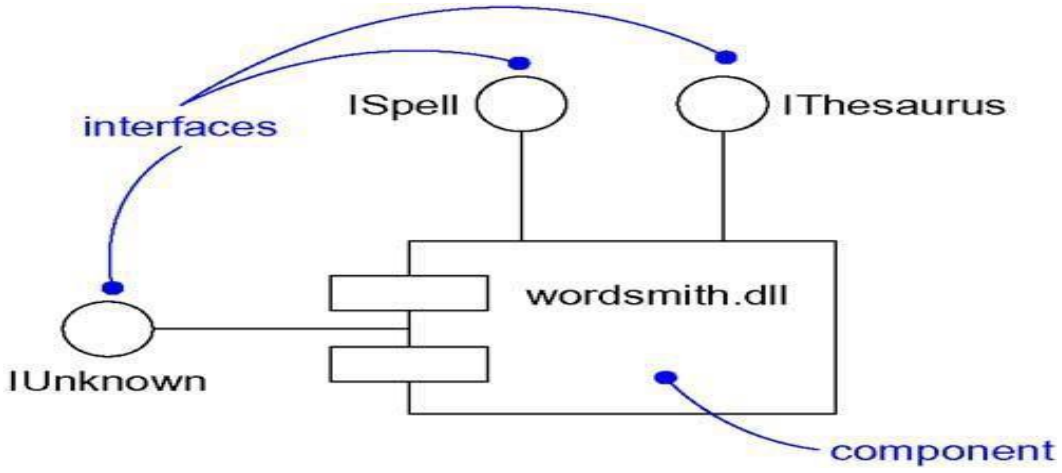



**Fig: Interfaces**

**Terms and Concepts Interface**

- An interface is a collection of operations that are used to specify a service of a class or a component

**Type**

- A type is a stereotype of a class used to specify a domain of objects, together with the operations (but not the methods) applicable to the object.

**Role**

- A role is the behavior of an entity participating in a particular context.

An interface may be rendered as a stereotyped class in order to expose its operations and other properties.

**Names**

- Every interface must have a name that distinguishes it from other interfaces.
- A name is a textual string. That name alone is known as a simple name;
- A path name is the interface name prefixed by the name of the package

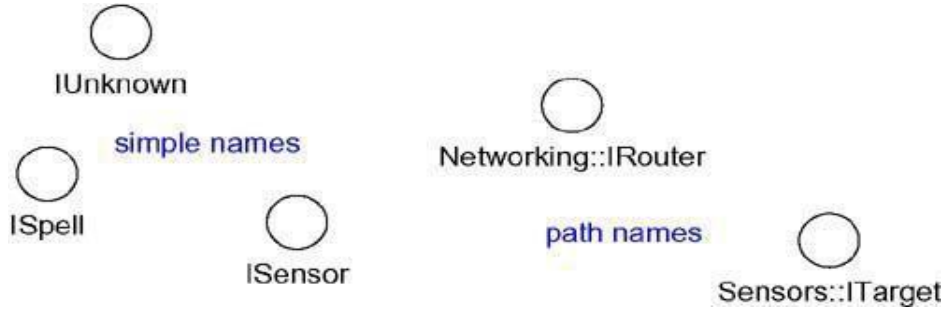



**Fig: Simple and Path Names**

**Operations**

- An interface is a named collection of operations used to specify a service of a class or of a component.
- Unlike classes or types, interfaces do not specify any structure (so they may not include any attributes), nor do they specify any implementation
- These operations may be adorned with visibility properties, concurrency properties, stereotypes, tagged values, and constraints.
- you can render an interface as a stereotyped class, listing its operations in the appropriate compartment. Operations may be drawn showing only their name, or they may be augmented to show their full signature and other properties.

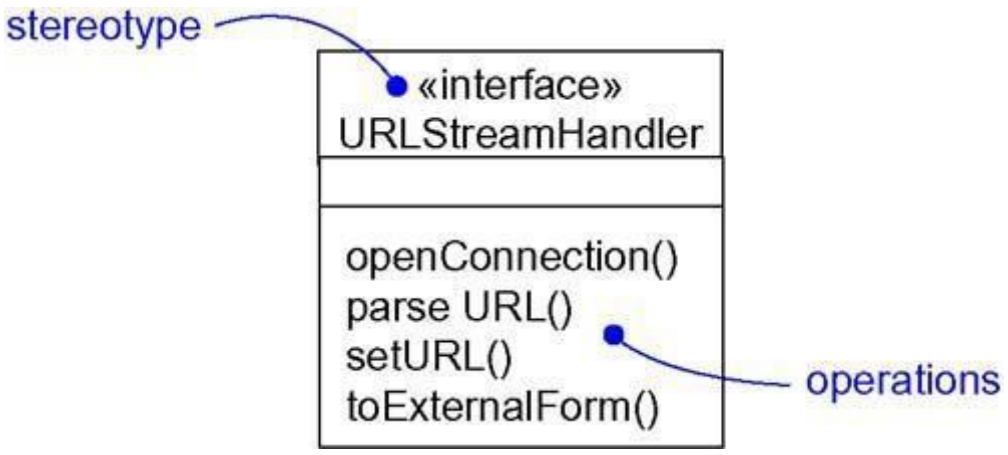



**Fig: Operations**

**Relationships**

- Like a class, an interface may participate in generalization, association, and dependency relationships. In addition, an interface may participate in realization relationships.
- An interface specifies a contract for a class or a component without dictating its implementation. A class or component may realize many interfaces.
- We can show that an element realizes an interface in two ways.
- First, you can use the simple form in which the interface and its realization relationship are rendered as a lollipop sticking off to one side of a class or component.
- Second, you can use the expanded form in which you render an interface as a stereotyped class, which allows you to visualize its operations and other properties, and then draw a realization relationship from the classifier or component to the interface.

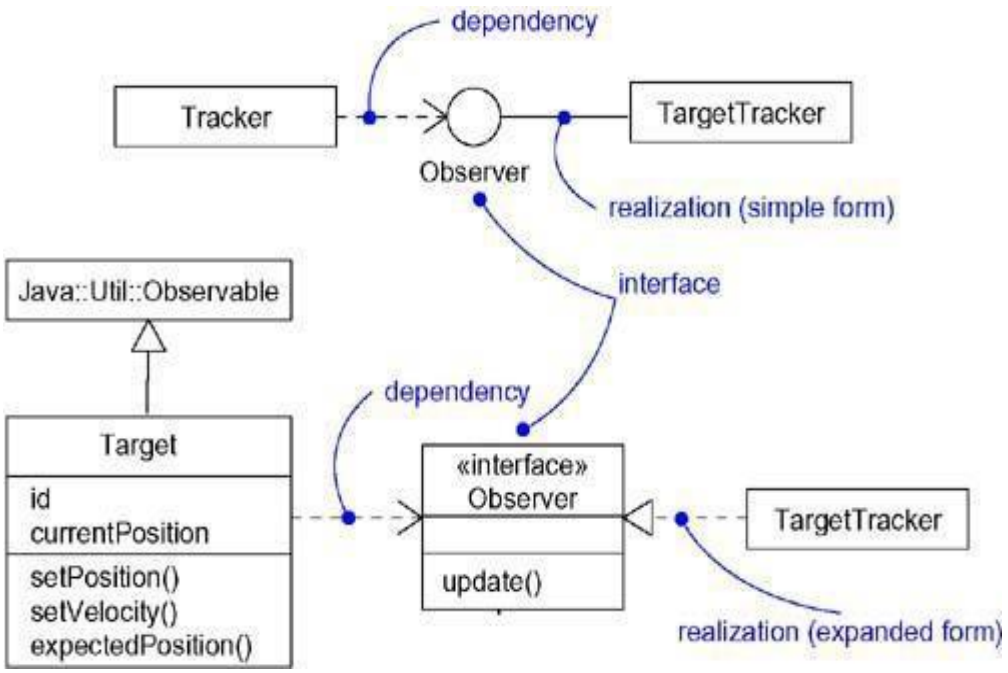


**Fig: Realizations Understanding an Interface**

- In the UML, you can supply much more information to an interface in order to make it understandable and approachable.
- First, you may attach pre- and post-conditions to each operation and invariants to the class or component as a whole. By doing this, a client who needs to use an interface will be able to understand what the interface does and how to use it, without having to dive into an implementation.
- We can attach a state machine to the interface. You can use this state machine to specify the legal partial ordering of an interface's operations.
- We can attach collaborations to the interface. You can use collaborations to specify the expected behavior of the interface through a series of interaction diagrams.

**Types and Roles**

- A role names a behavior of an entity participating in a particular context. Stated another way, a role is the face that an abstraction presents to the world.
- For example, consider an instance of the class Person. Depending on the context, that Person instance may play the role of Mother, Comforter, PayerOfBills, Employee, Customer, Manager, Pilot, Singer, and so on. When an object plays a particular role, it presents a face to the world, and clients that interact with it expect a certain behavior depending on the role that it plays at the time.
- An instance of Person in the role of Manager would present a different set of properties than if the instance were playing the role of Mother.
- In the UML, you can specify a role an abstraction presents to another abstraction by adorning the name of an association end with a specific interface.

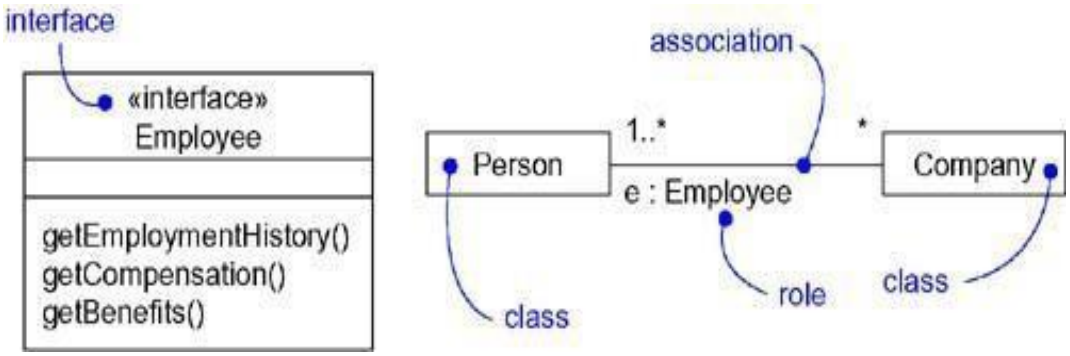



**Fig: Roles**

- A class diagram like this one is useful for modeling the static binding of an abstraction to its interface. You can model the dynamic binding of an abstraction to its interface by using the become stereotype in an interaction diagram, showing an object changing from one role to another.
- If you want to formally model the semantics of an abstraction and its conformance to a specific interface, you'll want to use the defined stereotype type
- Type is a stereotype of class, and you use it to specify a domain of objects, together with the operations (but not the methods) applicable to the objects of that type. The concept of type is closely related to that of interface, except that a type's definition may include attributes while an interface may not.

**Common Modeling Techniques Modeling the Seams in a System**

- The most common purpose for which you'll use interfaces is to model the seams in a system composed of software components, such as COM+ or Java Beans.
- Identifying the seams in a system involves identifying clear lines of demarcation in your architecture. On either side of those lines, you'll find components that may change independently, without affecting the components on the other side,

**To Modeling the Seams in a System**

- Within the collection of classes and components in your system, draw a line around those that tend to be tightly coupled relative to other sets of classes and components.

- Refine your grouping by considering the impact of change. Classes or components that tend to change together should be grouped together as collaborations.
- Consider the operations and the signals that cross these boundaries, from instances of one set of classes or components to instances of other sets of classes and components.
- Package logically related sets of these operations and signals as interfaces.
- For each such collaboration in your system, identify the interfaces it relies on (imports) and those it provides to others (exports). You model the importing of interfaces by dependency relationships, and you model the exporting of interfaces by realization relationships.
- For each such interface in your system, document its dynamics by using pre- and post-conditions for each operation, and use cases and state machines for the interface as a whole.
- The below figure shows the seams surrounding a component (the library ledger.dll) drawn from a financial system. This component realizes three interfaces: IUnknown, ILedger, and IReports. In this diagram, IUnknown is shown in its expanded form; the other two are shown in their simple form, as lollipops. These three interfaces are realized by ledger.dll and are exported to other components for them to build on.
- As this diagram also shows, ledger.dll imports two interfaces, IStreaming and ITransaction, the latter of which is shown in its expanded form. These two interfaces are required by the ledger.dll component for its proper operation. Therefore, in a running system, you must supply components that realize these two interfaces.
- By identifying interfaces such as ITransaction, you've effectively decoupled the components on either side of the interface, permitting you to employ any component that conforms to that interface.

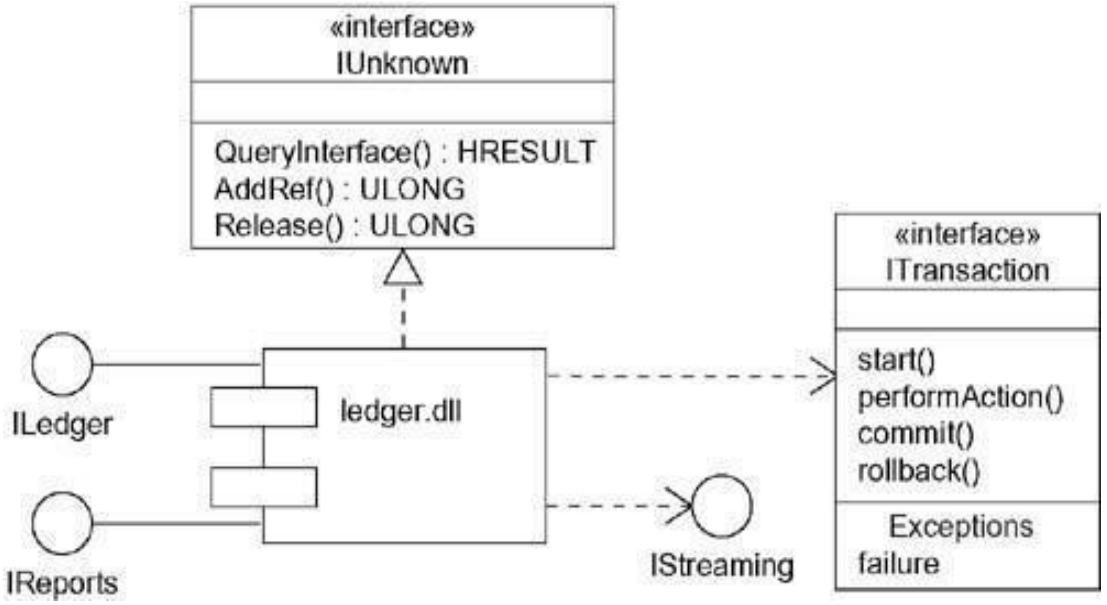
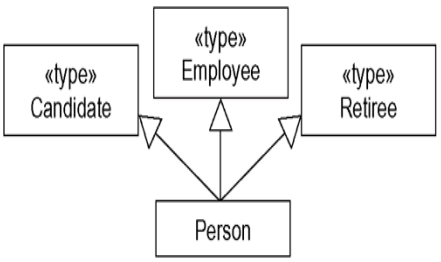



**Fig: Modeling the Seams in a System Modeling Static and Dynamic Types**

- Most object-oriented programming languages are statically typed, which means that the type of an object is bound at the time the object is created.
- Even so, that object will likely play different roles over time.
- Modeling the static nature of an object can be visualized in a class diagram. However, when you are modeling things like business objects, which naturally change their roles throughout a workflow,

**To model a dynamic type**

- Specify the different possible types of that object by rendering each type as a class stereotyped as type (if the abstraction requires structure and behavior) or as interface (if the abstraction requires only behavior).
- Model all the roles the of the object may take on at any point in time. You can do so in two ways:
    a. First, in a class diagram, explicitly type each role that the class plays in its association with other classes. Doing this specifies the face instances of that class put on in the context of the associated object.
    b. Second, also in a class diagram, specify the class-to-type relationships using generalization.
- In an interaction diagram, properly render each instance of the dynamically typed class. Display the role of the instance in brackets below the object's name.
- To show the change in role of an object, render the object once for each role it plays in the interaction, and connect these objects with a message stereotyped as become.



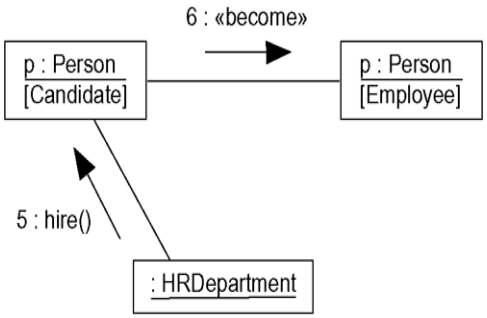




**Fig: Modeling Static Types** **Fig: Modeling Dynamic Types Package**

- A package is a general-purpose mechanism for organizing elements into groups. Graphically, a package is rendered as a tabbed folder.
- The UML provides a graphical representation of package, This notation permits you to visualize groups of elements that can be manipulated as a whole and in a way that lets you control the visibility of and access to individual elements.

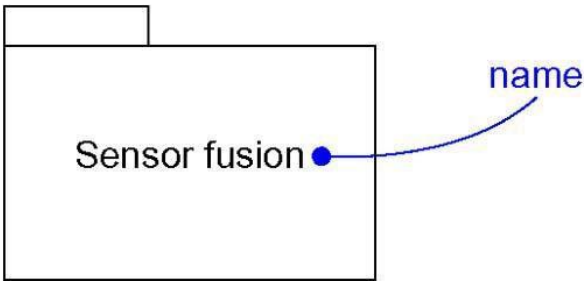



**Fig: Packages**

**Terms and Concepts**

- A package is a general-purpose mechanism for organizing elements into groups. Graphically, a package is rendered as a tabbed folder.

**Names**

- Every package must have a name that distinguishes it from other packages. A name is a textual string.
- That name alone is known as a simple name; a path name is the package name prefixed by the name of the package in which that package lives
- We may draw packages adorned with tagged values or with additional compartments to expose their details.

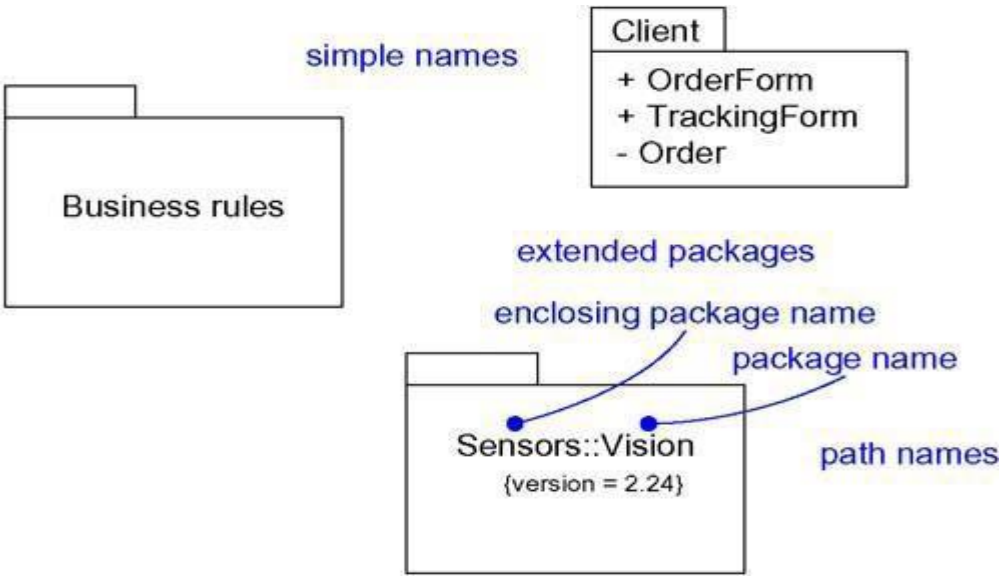



**Fig: Simple and Extended Package**

**Owned Elements**

- A package may own other elements, including classes, interfaces, components, nodes, collaborations, use cases, diagrams, and even other packages.
- Owning is a composite relationship, which means that the element is declared in the package. If the package is destroyed, the element is destroyed. Every element is uniquely owned by exactly one package.
- Elements of different kinds may have the same name within a package. Thus, you can have a class named Timer, as well as a component named Timer, within the same package.
- Packages may own other packages. This means that it's possible to decompose your models hierarchically.
- We can explicitly show the contents of a package either textually or graphically.

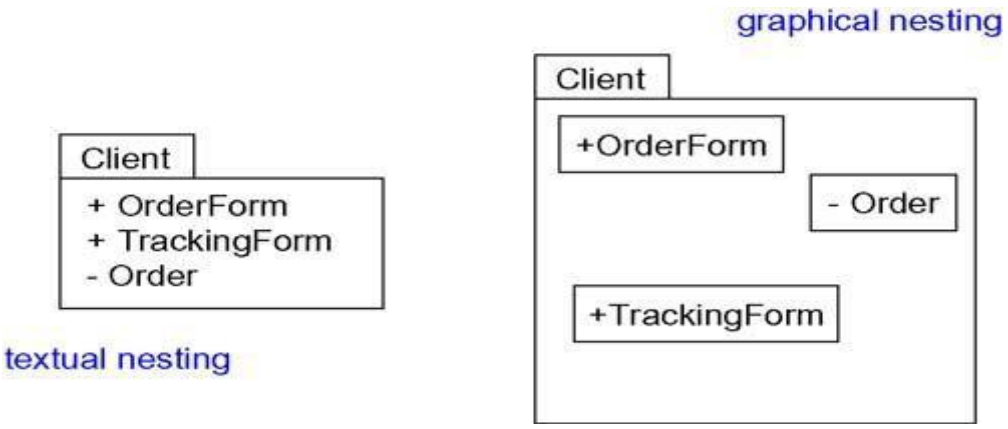



**Fig: Owned Elements**

**Visibility**

- You can control the visibility of the elements owned by a package just as you can control the visibility of the attributes and operations owned by a class.
- Typically, an element owned by a package is public, which means that it is visible to the contents of any package that imports the element's enclosing package.
- Conversely, protected elements can only be seen by children, and private elements cannot be seen outside the package in which they are declared.

- We specify the visibility of an element owned by a package by prefixing the element's name with an appropriate visibility symbol.

**Importing and Exporting**

- In the UML, you model an import relationship as a dependency adorned with the stereotype import
- Actually, two stereotypes apply here—import and access— and both specify that the source package has access to the contents of the target.
- Import adds the contents of the target to the source's namespace
- Access does not add the contents of the target
- The public parts of a package are called its exports.
- The parts that one package exports are visible only to the contents of those packages that explicitly import the package.
- Import and access dependencies are not transitive

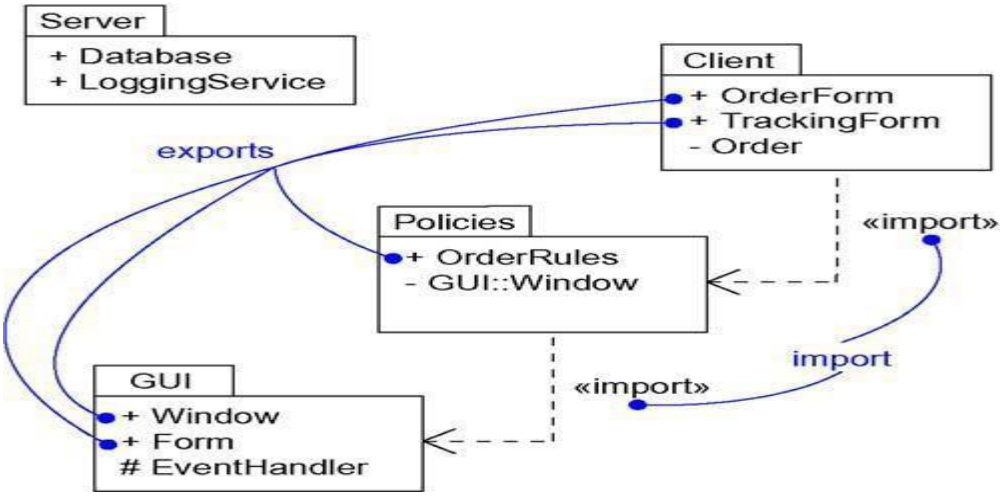



**Fig: Importing and Exporting**

**Generalization**

- There are two kinds of relationships you can have between packages: import and access dependencies used to import into one package elements exported from another and generalizations, used to specify families of packages
- Generalization among packages is very much like generalization among classes

- Packages involved in generalization relationships follow the same principle of substitutability as do classes. A specialized package (such as WindowsGUI) can be used anywhere a more general package (such as GUI) is used

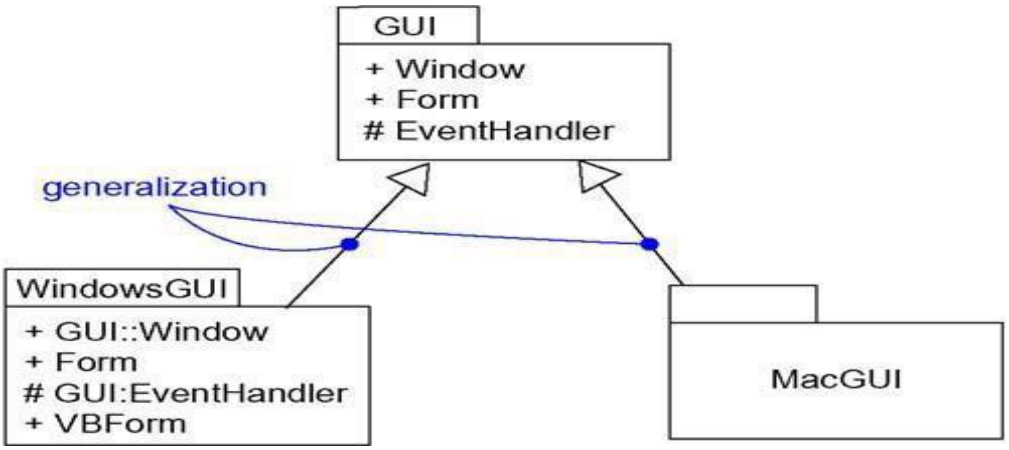



**Fig: Generalization among Packages**

**Standard Elements**

- All of the UML's extensibility mechanisms apply to packages. Most often, you'll use tagged values to add new package properties (such as specifying the author of a package) and stereotypes to specify new kinds of packages (such as packages that encapsulate operating system services).
- The UML defines five standard stereotypes that apply to packages.

| | |
|---|---|
| 1. facade | Specifies a package that is only a view on some other package |
| 2. framework | Specifies a package consisting mainly of patterns |
| 3. stub | Specifies a package that serves as a proxy for the public contents |

| | |
|---|---|
| | of another package |
| 4. subsystem | Specifies a package representing an independent part of the entire<br>system being modeled |
| 5. system | Specifies a package representing the entire system being modeled |

The UML does not specify icons for any of these stereotypes

**Common Modeling Techniques Modeling Groups of Elements**

- The most common purpose for which you'll use packages is to organize modeling elements into groups that you can name and manipulate as a set.
- There is one important distinction between classes and packages:
- Packages have no identity (meaning that you can't have instances of packages, so they are invisible in the running system); classes do have identity (classes have instances, which are elements of a running system).

**To model groups of elements**

- Scan the modeling elements in a particular architectural view and look for clumps defined by elements that are conceptually or semantically close to one another.
- Surround each of these clumps in a package.
- For each package, distinguish which elements should be accessible outside the package. Mark them public, and all others protected or private. When in doubt, hide the element.
- Explicitly connect packages that build on others via import dependencies
- In the case of families of packages, connect specialized packages to their more general part via generalizations.

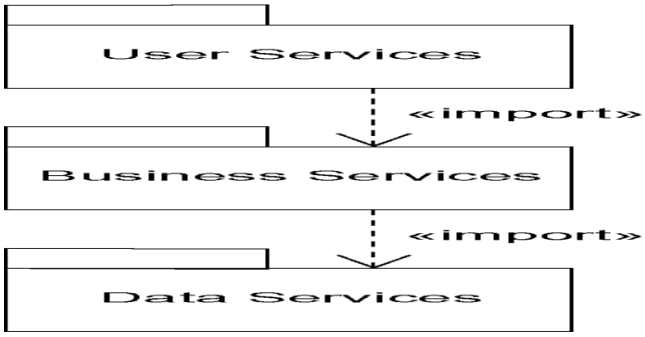



**Fig: Modeling Groups of Elements**

**Modeling Architectural Views**

- We can use packages to model the views of an architecture.
- Remember that a view is a projection into the organization and structure of a system, focused on a particular aspect of that system.
- This definition has two implications. First, you can decompose a system into almost orthogonal packages, each of which addresses a set of architecturally significant decisions.( design view, a process view, an implementation view, a deployment view, and a use case view)
- Second, these packages own all the abstractions germane to that view.(Implementation view)

**To model architectural views**

- Identify the set of architectural views that are significant in the context of your problem. In practice, this typically includes a design view, a process view, an implementation view, a deployment view, and a use case view.
- Place the elements (and diagrams) that are necessary and sufficient to visualize, specify, construct, and document the semantics of each view into the appropriate package.
- As necessary, further group these elements into their own packages.
- There will typically be dependencies across the elements in different views. So, in general, let each view at the top of a system be open to all others at that level.

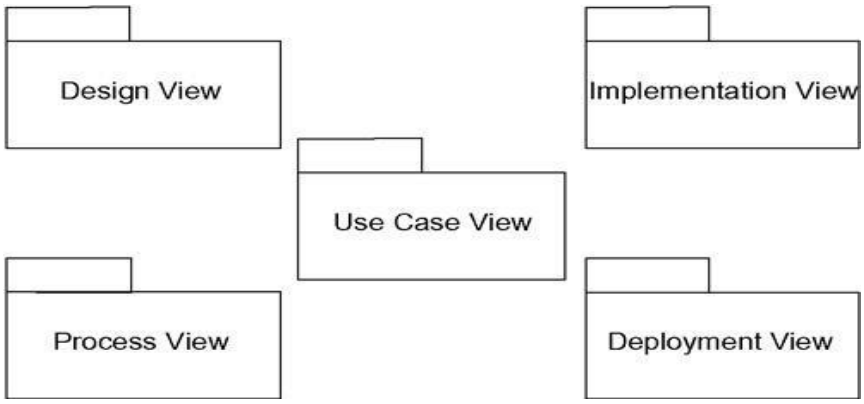



**Fig: Modeling Architectural Views**